

Module 4: Project Estimation & Scheduling

Albrecht Function Points (FP), Boehm's COCOMO I Equations & PERT / CPM Calculations

Module IV: Verification, Validation & Testing — Complete 16-Topic Syllabus Tracker

Topic 40: Verification ("Are We Building the Product Right?")

Topic 42: V&V Planning & Test Strategy Life Cycle

Topic 44: Static Analysis (Control Flow, Data Flow & Linters)

Topic 46: Testing Functions & Test Harness Design

Topic 48: White-Box Testing (Statement & Branch Coverage)

Topic 50: Basis Path Testing & McCabe's Cyclomatic Complexity

Topic 52: Unit Testing (Stubs, Drivers & Test Doubles)

Topic 54: System Testing (Stress, Load, Recovery & Acceptance)

Topic 41: Validation ("Are We Building the Right Product?")

Topic 43: Software Inspection (Fagan Formal Defect Reviews)

Topic 45: Software Testing Foundations & Defect Taxonomies

Topic 47: Test Case Design (Formal Test Case Artifact Structure)

Topic 49: Black-Box Testing (Equivalence Partitioning & BVA)

Topic 51: Control Structure Testing (Condition & Loop Testing)

Topic 53: Integration Testing (Top-Down, Bottom-Up, Sandwich)

Topic 55: Software Reliability & Availability (MTBF, MTTF, MTTR)

Topic 40 – 44: Verification vs. Validation & Static Quality Techniques

Parameter	Verification (Static Quality Assurance)	Validation (Dynamic Quality Control)
Boehm's Question	"Are we building the product right?"	"Are we building the right product?"
Execution	Static evaluation <i>without executing code</i> (Inspections, Walkthroughs, Linters).	Dynamic execution of software using actual test cases.
Target Evaluation	Verifies whether design/code conforms to SRS specifications.	Validates whether the operational system satisfies actual user needs.

Topic 48 & 49: White-Box vs. Black-Box Testing Methodologies

Feature	Black-Box Testing (Functional)	White-Box Testing (Structural / Glass-Box)
Knowledge Needed	Zero internal code knowledge; tests external inputs vs. expected outputs.	Full knowledge of source code logic, branches, loops, and data structures.
Primary Techniques	• Equivalence Partitioning (EP): Divides inputs into valid/invalid equivalence classes. • Boundary Value Analysis (BVA): Tests boundaries (min, min + 1, nominal, max – 1, max).	• Statement Coverage, Branch Coverage, Condition Coverage. • Basis Path Testing via Control Flow Graphs (CFG).

Topic 50: Basis Path Testing & McCabe's Cyclomatic Complexity

M McCabe's Cyclomatic Complexity $V(G)$ Formulations (McCabe, 1976): 1. From Edges (E) and Nodes (N):

$V(G) = E - N + 2$ 2. From Predicate / Decision Nodes (P): $V(G) = P + 1$ 3. From Enclosed Regions (R): $V(G) = R$

Significance: $V(G)$ defines the exact number of linearly independent execution paths required to achieve 100% branch coverage!

Step-by-Step Solved Problem: Cyclomatic Complexity & Independent Paths

```
int find_max(int a, int b, int c) {  
    int max = a;           // Node 1  
    if (b > max)            // Node 2 (Predicate)  
        max = b;           // Node 3  
    if (c > max)            // Node 4 (Predicate)  
        max = c;           // Node 5  
    return max;            // Node 6  
}
```

Graph Analysis: Nodes $N = 6$, Edges $E = 7$, Predicates $P = 2$ (Nodes 2 and 4).

$$V(G) = E - N + 2 = 7 - 6 + 2 = \mathbf{3} \quad (\text{or } P + 1 = 2 + 1 = \mathbf{3})$$

Set of 3 Linearly Independent Paths:

- **Path 1:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$ (Test: $a = 10, b = 5, c = 2$)
- **Path 2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$ (Test: $a = 5, b = 10, c = 2$)
- **Path 3:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6$ (Test: $a = 5, b = 2, c = 10$)

Topic 52 – 55: Levels of Testing & Software Reliability

Software Reliability & Availability Formulas:

Mean Time Between Failures (MTBF) = Mean Time To Failure (MTTF) + Mean Time To Repair (MTTR)

$$\text{Availability (A)} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

M4 Active Recall & Exam Questions

Q1. Explain Top-Down vs. Bottom-Up Integration Testing. Define Stubs and Drivers with diagrams. (10 Marks)

- **Top-Down Integration:** Modules are integrated from top to bottom of the control hierarchy. Subordinate modules not yet developed are simulated using **Stubs** (dummy subroutines called by the tested module).
- **Bottom-Up Integration:** Low-level worker modules are tested first. Superordinate modules not yet developed are simulated using **Drivers** (test harnesses that invoke the module and display results).
- **Sandwich (Hybrid) Integration:** Combines top-down for top layers and bottom-up for low-level utility libraries, minimizing stub/driver development overhead.